

High-Level Design (HLD)

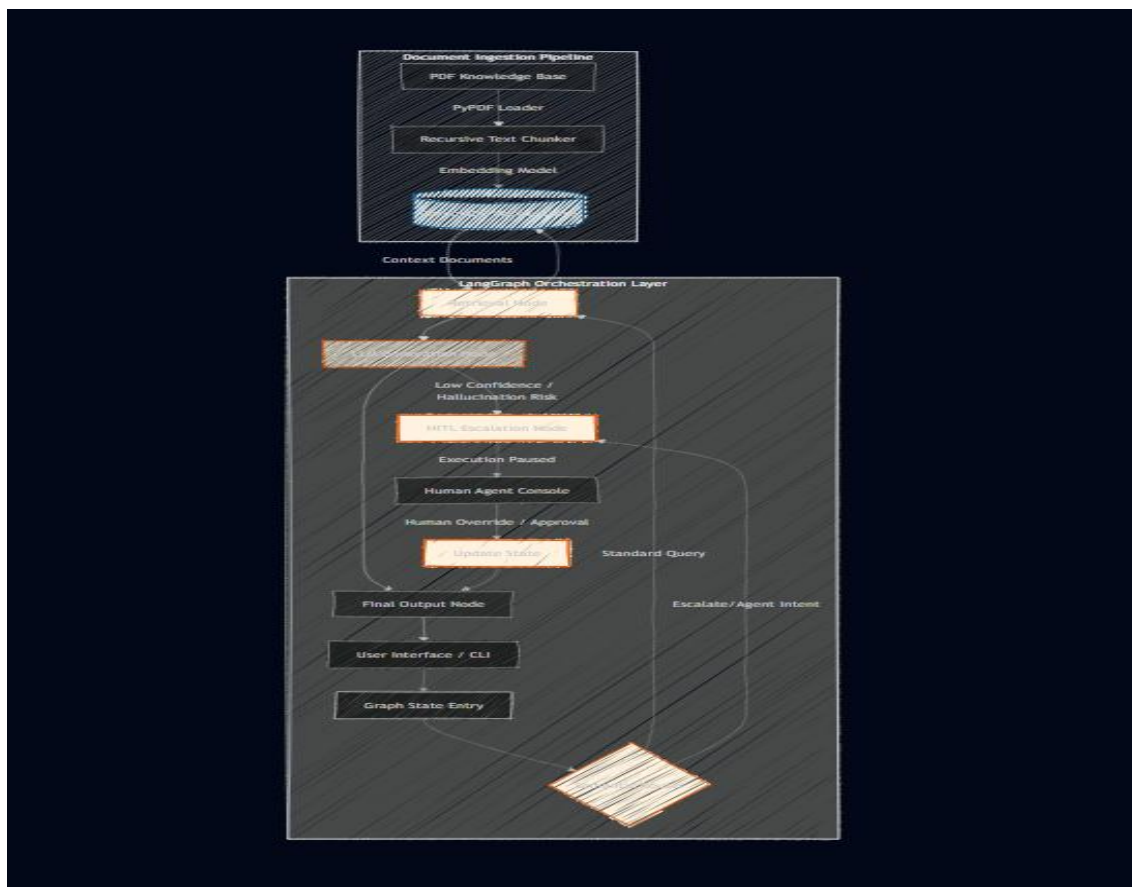
Project: RAG-Based Customer Support Assistant with LangGraph & HITL

1. System Overview

Problem Definition: Customer support teams are frequently bottlenecked by repetitive queries that can be resolved by referencing existing documentation. Traditional chatbots lack context-awareness, and standard LLMs hallucinate when asked company-specific questions. Furthermore, fully autonomous AI systems pose a risk when handling highly complex or sensitive customer escalations without human oversight.

Scope of the System: This system implements a state-aware Retrieval-Augmented Generation (RAG) architecture orchestrated by LangGraph. It ingests PDF knowledge bases, retrieves semantic matches for user queries, and generates contextual answers. Crucially, it introduces a decision-making routing layer that can autonomously resolve standard queries or pause execution and escalate to a Human-in-the-Loop (HITL) for complex, sensitive, or low-confidence scenarios.

2. Architecture Diagram



3. Component Description

- Document Loader: Utilizes PyPDFLoader to extract raw text content and metadata (page numbers, document names) from the source knowledge base.
- Chunking Strategy: RecursiveCharacterTextSplitter. Text is broken down into manageable chunks (e.g., 512 tokens) with a sliding overlap (e.g., 50 tokens) to ensure context isn't lost at the hard boundaries of paragraphs.
- Embedding Model: HuggingFaceBgeEmbeddings (specifically BAAI/bge-small-en-v1.5). Chosen for its high retrieval accuracy on the MTEB leaderboard while remaining computationally lightweight.
- Vector Store: ChromaDB. An embedded, open-source vector database used to store the dense vector representations of the document chunks and perform semantic similarity searches.
- Retriever: A Vector Store Retriever utilizing Maximal Marginal Relevance (MMR) search. MMR ensures the retrieved documents are not only highly relevant to the query but also diverse, reducing redundant context.
- LLM: Llama-3.1-8b hosted via Groq. Provides sub-second inference latency and excellent JSON-formatting adherence for structured routing decisions.
- Graph Workflow Engine: LangGraph. Acts as the state machine managing the cyclical flow of data, enabling nodes to pass a shared State object back and forth.
- Routing Layer: An LLM-powered conditional edge within LangGraph. It analyzes the user query intent and outputs a strictly formatted JSON decision (e.g., "route": "retrieval" or "route": "escalate").
- HITL Module: Utilizes LangGraph's native interrupt_before functionality. When triggered, the graph suspends execution, awaits human input to modify the State object, and resumes execution seamlessly.

4. Data Flow

The system operates in two distinct phases:

Phase 1: Ingestion Lifecycle (Offline)

1. PDF documents are parsed into raw text strings.
2. Text is split into 512-token chunks.
3. The Embedding Model maps each chunk to a high-dimensional vector.
4. Vectors and their original text payloads are indexed into ChromaDB.

Phase 2: Query Lifecycle (Runtime)

1. Input: User submits a support question.

2. Routing: The Router Node evaluates the query. If the query contains aggressive sentiment or explicitly asks for a human, it branches to the HITL node. Otherwise, it branches to the Retrieval Node.
3. Retrieval: The system embeds the user query, searches ChromaDB, and returns the top 3 most relevant chunks.
4. Generation: The LLM synthesizes an answer using *only* the retrieved context. It also assigns a confidence score to its own answer.
5. Fallback (HITL): If the LLM confidence score is below the accepted threshold, execution halts and alerts a human.
6. Output: The final response (either LLM-generated or Human-provided) is delivered to the user.

5. Technology Choices

- Why Chroma DB? For an initial RAG system, ChromaDB runs locally in-memory or via SQLite. It removes the need for cloud infrastructure provisioning (like Pinecone or AWS OpenSearch) while offering identical semantic search capabilities.
- Why LangGraph? Traditional RAG pipelines (like LangChain's `create_retrieval_chain`) are Directed Acyclic Graphs (DAGs)—they move in one direction. Support bots require loops (e.g., asking clarifying questions) and pauses (HITL). LangGraph is explicitly designed to handle stateful, multi-actor workflows.
- LLM Choice (Groq / Llama-3.1): Customer support requires immediate responses. By utilizing Groq's LPU (Language Processing Unit) architecture, the system minimizes Time-To-First-Token (TTFT), making the routing layer virtually instantaneous.

6. Scalability Considerations

If this system were to be deployed to 100,000 users, the architecture would need the following adjustments:

- Handling Large Documents: Local ChromaDB would become a memory bottleneck. The system would migrate to a managed vector database like Pinecone, Qdrant, or Milvus. Document ingestion would be decoupled into an asynchronous message queue (e.g., Celery/RabbitMQ) or handled via Apache Spark.
- Increasing Query Load: The LangGraph execution environment would need to be deployed behind a load balancer (e.g., via LangServe or FastAPI) using stateless containers (Docker/Kubernetes). Graph state persistence would migrate from local memory to an external Redis or Postgres database (using LangGraph's checkpointing).
- Latency Concerns: To prevent the LLM routing layer from adding latency to every single query, a fast Semantic Cache (e.g., RedisVL) would be implemented before the router to instantly return answers to historically common questions (e.g., "What are your business hours?").